Колледж ТОО «Astana IT University»

КУРСОВОЙ ПРОЕКТ

по профессиональному модулю ПМ07
«Разработка мобильных приложений»

на тему:
«Разработка мобильного приложения StudyFlow для планирования учебных задач студентов колледжа»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Выполнил:
студент группы ПО2408
Айткужинов Амир
_____ (подпись)

Проверил:
преподаватель
Жаңабай Айбар Нұрғалиұлы
_____ (подпись)

Оценка: _____
«____» _____ 2026 г.

Астана 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....6
1.1 Анализ предметной области.....6
1.2 Особенности мобильных приложений.....7
1.3 Принципы разработки мобильных приложений.....8
1.4 Анализ похожих решений.....10
1.5 Обоснование выбора технологий.....11
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....14
2.1 Требования к приложению.....14
2.2 Структура приложения и навигация.....14
2.3 Модель данных и хранение.....16
2.4 Экран запуска.....17
2.5 Экран регистрации.....18
2.6 Экран авторизации и связь с регистрацией.....21
2.7 Главная страница и навигация.....22
2.8 Основной раздел «Мои задачи».....24
2.9 Профиль и выход из аккаунта.....25
2.10 Тестирование приложения.....27
ЗАКЛЮЧЕНИЕ.....29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....30
ПРИЛОЖЕНИЕ А. Фрагменты исходного кода.....32

ВВЕДЕНИЕ

Учебный процесс студента колледжа одновременно включает несколько дисциплин, и по каждой из них выдаются домашние задания, лабораторные и практические работы, рефераты, курсовые проекты, назначаются контрольные и зачёты. Сроки сдачи устанавливают разные преподаватели, сообщаются они устно на занятии, в групповых чатах или в электронном журнале и нередко совпадают по времени. В результате студент тратит время на поиск информации о заданиях, плохо представляет общий объём оставшейся работы и пропускает часть сроков.

Смартфон — устройство, которое у студента всегда под рукой. По данным аналитического отчёта Digital 2026: Kazakhstan, в конце 2025 года интернетом в Казахстане пользовались 19,5 млн человек (93,4 % населения), а число подключений сотовой связи составило 131 % от численности населения [13]. Поэтому наиболее удобной формой инструмента планирования учёбы является мобильное приложение: задание можно записать сразу, как только его объявили, а список ближайших сроков проверить в любой момент.

Распространённые планировщики (Google Tasks, Microsoft To Do, Todoist, TickTick) рассчитаны на широкий круг дел, требуют учётной записи стороннего сервиса, частично платные и не учитывают специфику учёбы: привязку задачи к предмету, наглядное выделение просроченных работ, прогресс выполнения. Этим определяется актуальность разработки специализированного мобильного приложения для планирования учебных задач студента.

Цель проекта — разработать мобильное приложение StudyFlow, которое позволяет студенту колледжа зарегистрироваться, вести список учебных задач по предметам со сроками сдачи и приоритетами и отслеживать прогресс их выполнения.

Для достижения цели поставлены следующие задачи:

1) проанализировать предметную область и существующие решения для планирования задач;

2) изучить особенности и принципы разработки мобильных приложений, обосновать выбор технологий;

3) спроектировать структуру приложения, модель данных и схему навигации;

4) реализовать экран запуска, экраны регистрации и авторизации с проверкой вводимых данных;

5) реализовать сохранение данных пользователя, обработку состояния входа и функцию выхода;

6) реализовать главную страницу, основной раздел «Мои задачи» и раздел профиля;

7) протестировать приложение по полному сценарию работы — от запуска до выхода.

Объект разработки — мобильное приложение для планирования учебных задач студента.

Предмет разработки — функциональные возможности, пользовательский интерфейс и программная реализация приложения StudyFlow на фреймворке Flutter.

Целевая аудитория — студенты колледжей и вузов в возрасте 15–22 лет, которые пользуются смартфоном на Android или iOS и хотят держать под контролем сроки сдачи учебных работ. Приложение также подойдёт старшеклассникам и слушателям курсов.

Основной функционал приложения:

– регистрация с указанием имени, учебной группы, почты и пароля, авторизация и выход из аккаунта;

– главная страница с прогрессом выполнения задач и ближайшими дедлайнами;

– создание, редактирование, удаление и отметка выполнения учебных задач с указанием предмета, срока, приоритета и комментария;

– поиск и фильтрация задач;

– профиль пользователя со статистикой и возможностью изменить свои данные.

Практическая значимость работы состоит в том, что приложение можно использовать для ежедневного планирования учёбы: оно работает без подключения к интернету и не требует сторонних учётных записей. Кроме того, исходный код, собранный в одном файле и разделённый на логические разделы, может служить учебным примером реализации типовых экранов мобильного приложения на Flutter.

Пояснительная записка состоит из введения, теоретической и практической частей, заключения, списка использованных источников и приложения и оформлена с учётом требований ГОСТ 7.32-2017 [2].

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Предметной областью проекта является самостоятельная организация учебной деятельности студента. Учебная задача в рамках проекта — это конкретная работа, которую нужно выполнить к определённому сроку: домашнее задание, лабораторная или практическая работа, реферат, курсовой проект, подготовка к контрольной работе или экзамену.

Каждая учебная задача описывается набором характеристик:

– название — что именно нужно сделать;

– дисциплина — к какому предмету относится задание;

– срок сдачи — дата, к которой работа должна быть выполнена;

– приоритет — насколько задача важна относительно других;

– статус — выполнена задача или ещё нет;

– комментарий — требования преподавателя, номер варианта и другие пояснения.

Анализ типичного учебного процесса позволяет выделить проблемы, с которыми сталкивается студент:

– задания поступают из разных источников (устно на занятии, в групповом чате, в электронном журнале) и не собраны в одном месте;

– сроки по разным дисциплинам пересекаются, особенно в конце семестра;

– без наглядного списка трудно оценить объём оставшейся работы и вовремя заметить приближающийся или уже пропущенный срок;

– бумажный ежедневник неудобен для поиска и никак не выделяет просроченные работы.

Из этого следуют требования к программному решению: быстрый ввод задачи с привязкой к предмету и сроку; автоматическая сортировка задач по срокам; выделение задач, срок которых наступает сегодня или завтра либо уже прошёл; отметка выполнения и наглядный прогресс; поиск и

фильтрация. Список задач носит личный характер, поэтому каждый пользователь должен видеть только свои данные, а значит, в приложении необходимы регистрация и авторизация.

При регистрации приложение получает имя, адрес электронной почты и учебную группу пользователя, то есть персональные данные. Их сбор и обработка допускаются с согласия субъекта в соответствии с Законом Республики Казахстан «О персональных данных и их защите» [1], поэтому в форму регистрации включено обязательное согласие на обработку персональных данных.

1.2 Особенности мобильных приложений

Мобильное приложение — это программа, предназначенная для работы на смартфонах и планшетах. В отличие от настольных и веб-приложений она работает в особых условиях, которые необходимо учитывать при проектировании [6; 9]:

– небольшой экран: интерфейс строится в одну колонку, на экране размещается только главное, а второстепенные функции выносятся на отдельные экраны;

– сенсорный ввод: элементы управления должны быть достаточно крупными — рекомендуемый минимальный размер области касания составляет 48×48 dp в Material Design [16] и 44×44 pt в рекомендациях Apple [11]; ввод текста с экранной клавиатуры неудобен, поэтому там, где это возможно, вместо ввода используется выбор из вариантов;

– короткие сеансы работы: пользователь открывает приложение на несколько секунд, например чтобы записать задание на перемене, поэтому основное действие должно выполняться за минимальное число шагов;

– нестабильное подключение к сети: приложение должно оставаться работоспособным без интернета, поэтому данные хранятся локально на устройстве;

– жизненный цикл: операционная система может приостановить или завершить приложение в любой момент, поэтому данные сохраняются сразу после каждого изменения, а не при закрытии;

– ограниченные ресурсы: объём памяти и заряд батареи ограничены, лишние вычисления и запросы нежелательны;

– разнообразие устройств: приложение должно корректно отображаться на экранах разного размера и работать как на Android, так и на iOS.

По способу реализации выделяют три вида мобильных приложений:

1) нативные — разрабатываются отдельно под каждую платформу на её основном языке (Kotlin для Android, Swift для iOS); обеспечивают максимальную производительность и полный доступ к возможностям системы, но требуют двух отдельных кодовых баз;

2) кроссплатформенные — пишутся один раз и собираются для нескольких платформ (Flutter, React Native, Kotlin Multiplatform); сокращают затраты на разработку при производительности, близкой к нативной;

3) веб-приложения и прогрессивные веб-приложения (PWA) — работают в браузере и не требуют установки, но имеют ограниченный доступ к функциям устройства.

Для учебного проекта, который должен работать на Android и iOS, оптимален кроссплатформенный подход: одна кодовая база при интерфейсе, не уступающем нативному.

1.3 Принципы разработки мобильных приложений

При разработке StudyFlow учитывались общепринятые принципы проектирования интерфейсов и программирования.

Ориентация на цели пользователя. Интерфейс проектируется исходя из задач человека, а не из внутреннего устройства программы [3]. Главная цель студента — быстро увидеть, что и к какому сроку нужно сделать, поэтому на главной странице сразу показываются прогресс и ближайшие дедлайны.

Простота и очевидность. Пользователь не должен задумываться, как выполнить действие [4]. Каждый экран решает одну задачу, кнопки подписаны глаголами: «Войти», «Создать аккаунт», «Добавить задачу».

Эвристики юзабилити Я. Нильсена [17]. В проекте применены следующие из них:

– видимость состояния системы: индикатор загрузки на кнопке входа, всплывающие сообщения «Задача добавлена», «Профиль обновлён»;

– соответствие реальному миру: даты на русском языке в привычном виде («15 октября · через 2 дня»);

– контроль и свобода пользователя: отмена удаления задачи, кнопка «Назад» на вложенных экранах;

– единообразие и стандарты: одна цветовая схема и компоненты Material Design 3 на всех экранах;

– предотвращение ошибок: проверка данных до отправки формы, подтверждение удаления и выхода;

– помощь в исправлении ошибок: сообщение выводится под конкретным полем и объясняет, как исправить ввод (например, «Формат: буквы и цифры, например ПО2408»).

Единый визуальный стиль. Используется система дизайна Material Design 3 [16]: цветовая схема строится от одного основного цвета, у полей ввода, кнопок и карточек одинаковые скругления и отступы.

Защита данных пользователя. Данные одного пользователя недоступны другому; при неверной авторизации приложение не уточняет, что именно введено неверно — почта или пароль, чтобы не подсказывать посторонним существующие адреса; после выхода закрытые разделы недоступны без повторного входа.

Качество программного кода. Код разделён на логические части: модели данных, хранилище, проверка ввода, экраны и общие виджеты; имена классов и переменных отражают их назначение, повторяющиеся элементы интерфейса вынесены в отдельные виджеты [5]. Учитывались также

требования Google к качеству Android-приложений [10]: корректная работа кнопки «Назад», отсутствие сбоев, сохранение данных пользователя, читаемость текста.

1.4 Анализ похожих решений

Для выявления сильных и слабых сторон существующих решений рассмотрены четыре распространённых приложения для планирования задач.

Google Tasks — простой список задач, встроенный в сервисы Google. Позволяет создавать списки, задачи со сроком и подзадачи. Требует учётной записи Google; учебной специфики (предметов, статистики выполнения) не имеет.

Microsoft To Do — планировщик компании Microsoft с разделом «Мой день», списками и напоминаниями. Требует учётной записи Microsoft; важные задачи можно только пометить отметкой «Важно», уровней приоритета нет.

Todoist — функциональный кроссплатформенный менеджер задач с проектами, метками, четырьмя уровнями приоритета и распознаванием дат в тексте. Часть возможностей доступна только в платном тарифе, а обилие функций требует времени на освоение.

TickTick — планировщик с календарём, таймером по методу «Помидора» и трекером привычек. Расширенные функции платные; для простого учёта домашних заданий интерфейс избыточен.

Результаты сравнения приведены в таблице 1.

Таблица 1 – Сравнение приложений для планирования задач

Критерий	Google Tasks	Microsoft To Do	Todoist	TickTick	StudyFlow
Ориентация на учебный процесс (предметы, группа)	нет	нет	нет	нет	да
Регистрация	аккаунт Google	аккаунт Microsoft	аккаунт Todoist	аккаунт TickTick	локальная, без внешних сервисов
Платные функции	нет	нет	есть	есть	нет
Уровни приоритета	нет	только «Важно»	4 уровня	есть	3 уровня
Статистика выполнения	нет	нет	есть	есть	есть
Сложность освоения	низкая	низкая	средняя	средняя	низкая

Анализ показал, что универсальные планировщики обладают широкими возможностями, но не ориентированы на учёбу, требуют учётной записи стороннего сервиса, а часть их функций платная. Нишей для StudyFlow является простое бесплатное приложение, работающее без сети и построенное вокруг предметов, сроков и прогресса. Из рассмотренных решений заимствованы удачные идеи: цветовая отметка приоритета (Todoist), подсчёт выполненных задач (TickTick) и короткая форма добавления задачи (Google Tasks).

1.5 Обоснование выбора технологий

Для выбора технологии разработки сравнивались нативный подход (Kotlin для Android и Swift для iOS), фреймворк React Native и фреймворк Flutter (таблица 2).

Таблица 2 – Сравнение технологий разработки мобильных приложений

Критерий	Нативная разработка	React Native	Flutter
Язык программирования	Kotlin / Swift	JavaScript / TypeScript	Dart
Кодовых баз для Android и iOS	две	одна	одна
Отрисовка интерфейса	компоненты платформы	компоненты платформы	собственный графический движок, одинаковый вид на всех платформах
Быстрое применение изменений без перезапуска	ограниченно	есть (Fast Refresh)	есть (Hot Reload)
Компоненты Material Design 3	на Android	через сторонние библиотеки	встроены в SDK
Сборка под веб для демонстрации	нет	через дополнительные библиотеки	встроена

Выбран фреймворк Flutter по следующим причинам:

– одна кодовая база для Android и iOS, а также возможность быстро проверить приложение в браузере;

– декларативное построение интерфейса из виджетов: экран описывается как функция от данных, и при изменении данных Flutter сам перерисовывает нужную часть [7; 14];

– встроенные виджеты Material Design 3, формы с проверкой ввода и навигация — приложению не нужны внешние библиотеки, что соответствует условиям задания;

– Hot Reload: изменения в коде применяются в работающем приложении за секунды, что ускоряет разработку и отладку интерфейса [8; 9];

– рабочий шаблон курсового проекта, выданный по модулю ПМ07, также написан на Flutter.

Dart — объектно-ориентированный язык со строгой типизацией и обязательной проверкой значений null (sound null safety), благодаря чему

многие ошибки обнаруживаются ещё при компиляции. Для разработки Dart компилируется «на лету» (JIT), что обеспечивает Hot Reload, а для выпуска приложения — заранее в машинный код процессора (AOT), что даёт высокую скорость работы [12]. Асинхронные операции, например чтение и запись файла, записываются с помощью async/await.

Для управления состоянием использован встроенный класс ChangeNotifier и виджет ListenableBuilder. Этот способ рекомендован документацией Flutter для приложений небольшого размера [15] и не требует сторонних пакетов (Provider, Bloc и др.).

Для хранения данных рассматривались пакеты shared_preferences, sqflite (SQLite), Hive и облачная база Firebase. Поскольку по условиям задания внешние библиотеки и подключение к сети не используются, данные сохраняются в JSON-файл средствами стандартных библиотек dart:io и dart:convert. Для учебного приложения этого достаточно; при развитии проекта хранилище можно заменить на SQLite без изменения экранов, так как работа с данными сосредоточена в одном классе.

Средства разработки: Flutter SDK (стабильная ветка; проект проверен на версии 3.47 с Dart 3.13), редактор Visual Studio Code или Android Studio с плагином Flutter, эмулятор Android или смартфон для запуска, браузер Chrome для быстрой проверки интерфейса.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Требования к приложению

На основе анализа предметной области и критериев задания сформулированы функциональные требования к приложению (таблица 3).

Таблица 3 – Функциональные требования и их реализация

№	Требование	Где реализовано
1	Экран запуска с логотипом и названием, автоматический переход дальше	SplashScreen
2	Регистрация с проверкой вводимых данных	RegisterScreen, Validators
3	Авторизация по почте и паролю, обработка неверного входа	LoginScreen, AppStore.login
4	Сохранение пользователей и состояния входа	AppStore (load, _save)
5	Главная страница с основной информацией	HomeTab
6	Нижняя навигация между разделами	HomeShell
7	Добавление, изменение, удаление и отметка задач	TasksTab, TaskFormScreen
8	Профиль пользователя и его редактирование	ProfileTab, EditProfileScreen
9	Выход из аккаунта с запретом доступа к закрытым разделам	confirmLogout, AppStore.logout

Нефункциональные требования:

– работа на Android и iOS, поддерживаемых текущей версией Flutter;

– работа без подключения к интернету и без внешних библиотек;

– интерфейс на русском языке в едином стиле Material Design 3;

– корректное отображение на экранах разной ширины: формы входа и регистрации ограничены по ширине 440 точками, а содержимое всех экранов прокручивается на маленьких экранах;

– сохранение данных сразу после каждого изменения.

2.2 Структура приложения и навигация

Приложение реализовано в одном файле lib/main.dart, разделённом на 15 пронумерованных разделов: оформление, работа с датами, модели данных, хранилище, проверка ввода, маршруты, экраны и общие виджеты.

Такое разбиение упрощает поиск нужного фрагмента кода. Перечень экранов приведён в таблице 4.

Таблица 4 – Экраны приложения

Экран	Класс	Как открывается	Назначение
Экран запуска	SplashScreen	маршрут «/»	заставка, загрузка данных
Авторизация	LoginScreen	маршрут «/login»	вход по почте и паролю
Регистрация	RegisterScreen	маршрут «/register»	создание аккаунта
Оболочка с вкладками	HomeShell	маршрут «/home»	нижняя навигация
Главная	HomeTab	вкладка 1	прогресс, ближайшие сроки
Мои задачи	TasksTab	вкладка 2	основной раздел
Профиль	ProfileTab	вкладка 3	данные пользователя, выход
Задача	TaskFormScreen	Navigator.push	добавление и изменение задачи
Редактирование профиля	EditProfileScreen	Navigator.push	изменение имени и группы

Основные экраны зарегистрированы как именованные маршруты в параметре routes виджета MaterialApp. Маршрут «/home» защищён: если в хранилище нет вошедшего пользователя, вместо главной открывается экран авторизации. Для переходов используются разные методы навигатора [14]:

– pushReplacementNamed — с экрана запуска: заставка заменяется следующим экраном, и кнопка «Назад» к ней не возвращает;

– pushNamedAndRemoveUntil — после входа и после выхода: вся история экранов очищается, поэтому после выхода нельзя вернуться в профиль кнопкой «Назад»;

– push и pop — для вложенных экранов (задача, редактирование профиля): экран открывается поверх текущего и при закрытии может вернуть результат, например текст сообщения «Задача добавлена».

Внутри оболочки HomeShell переключение разделов выполняет нижняя панель NavigationBar. Вкладки размещены в IndexedStack, который хранит

все три экрана и показывает выбранный, поэтому введённый текст поиска и выбранный фильтр не сбрасываются при переключении. Схема навигации показана на рисунке 1.

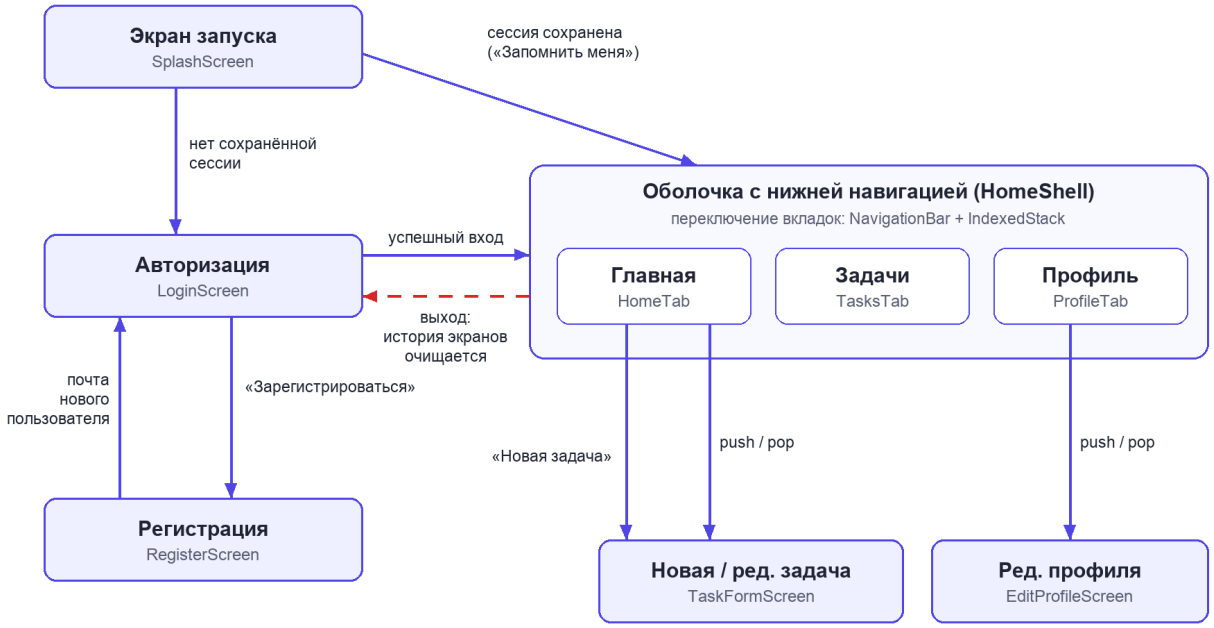


Рисунок 1 – Схема навигации приложения

2.3 Модель данных и хранение

Данные приложения описываются двумя классами. Класс AppUser хранит сведения о пользователе (таблица 5), класс StudyTask — об учебной задаче (таблица 6).

Таблица 5 – Поля класса AppUser

Поле	Тип	Описание
name	String	имя и фамилия
email	String	электронная почта (в нижнем регистре), служит логином
password	String	пароль
group	String	учебная группа — поле, добавленное в регистрацию
registeredAt	DateTime	дата регистрации

Таблица 6 – Поля класса StudyTask

Поле	Тип	Описание
id	String	уникальный идентификатор
ownerEmail	String	почта владельца: у каждого пользователя свой список
title	String	название задачи
subject	String	предмет
deadline	DateTime	срок сдачи
priority	int	приоритет: 0 — низкий, 1 — средний, 2 — высокий
note	String	комментарий
done	bool	отметка выполнения

Вся работа с данными сосредоточена в классе AppStore, который наследует ChangeNotifier. Он хранит словарь пользователей (ключ — почта), список задач, почту вошедшего пользователя и признак «Запомнить меня», а также содержит методы register, login, logout, updateProfile, addTask, updateTask, toggleTask, deleteTask и restoreTask. После любого изменения вызывается метод _commit: он уведомляет экраны через notifyListeners(), и они перерисовываются, а затем сохраняет данные в файл studyflow_data.json во временном каталоге приложения. При запуске метод load читает этот файл. В браузере доступа к файловой системе нет, поэтому там данные хранятся в памяти до перезагрузки страницы.

2.4 Экран запуска

Экран запуска (рисунок 2) выполнен в фирменных цветах приложения: градиентный фон от индиго к фиолетовому, белый логотип с изображением книги, название StudyFlow, подзаголовок «Планировщик учёбы студента», индикатор загрузки и подпись автора внизу экрана.

Появление логотипа анимировано: контроллер анимации AnimationController длительностью 1,2 с управляет плавным проявлением (FadeTransition) и увеличением (ScaleTransition) с кривой Curves.easeOutBack. Одновременно с анимацией метод _start загружает сохранённые данные. С помощью Future.wait приложение дожидается и загрузки, и минимального

времени показа заставки 2,5 с, после чего методом pushReplacementNamed открывает главную страницу, если сессия была сохранена, или экран авторизации.

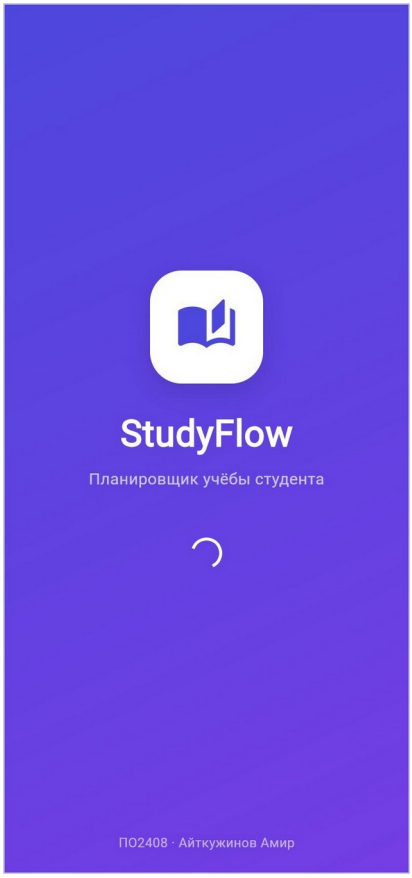


Рисунок 2 – Экран запуска

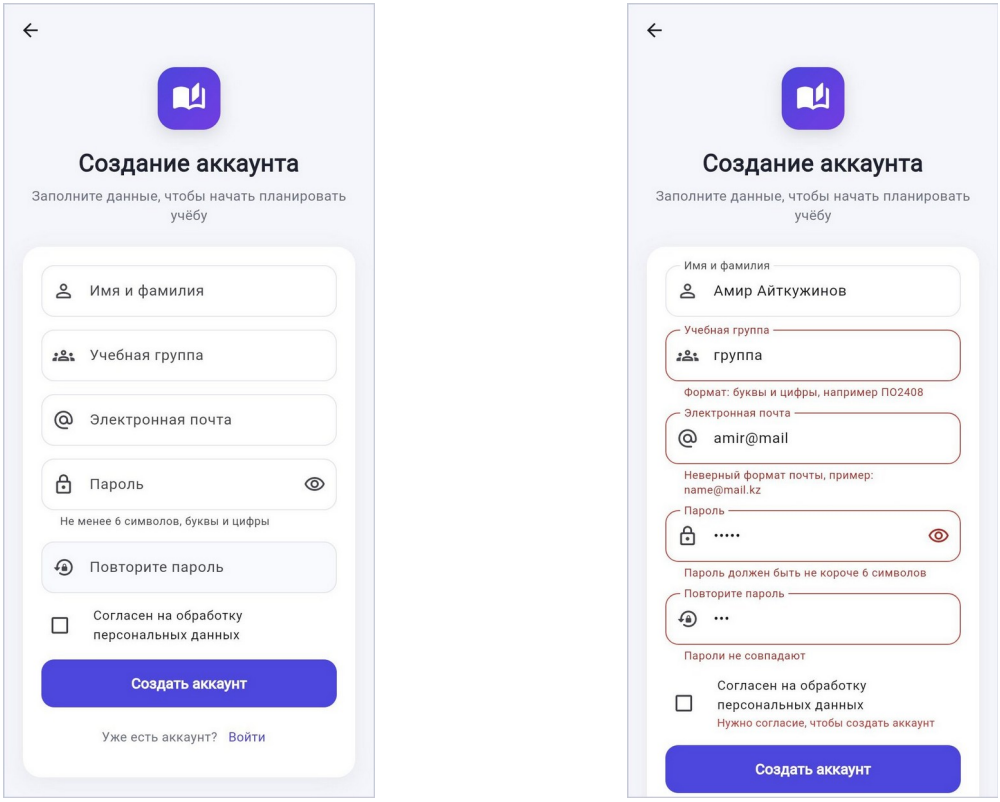
2.5 Экран регистрации

Форма регистрации содержит поля «Имя и фамилия», «Учебная группа», «Электронная почта», «Пароль», «Повторите пароль» и флажок согласия на обработку персональных данных (рисунок 3, а). Поля снабжены значками, у пароля есть кнопка показа и скрытия символов, под полем пароля выводится подсказка с требованиями. Правила проверки приведены в таблице 7.

Таблица 7 – Правила проверки полей регистрации

Поле	Правило	Сообщение при ошибке
Имя и фамилия	не пустое, не короче 2 символов, без цифр	«Введите имя и фамилию», «Имя не должно содержать цифры»
Учебная группа	от 1 до 5 букв, необязательный дефис и 3–4 цифры	«Формат: буквы и цифры, например ПО2408»
Электронная почта	соответствует формату name@domain.zone	«Неверный формат почты, пример: name@mail.kz»
Пароль	не короче 6 символов, есть буквы и цифры	«Пароль должен быть не короче 6 символов», «…содержать буквы и цифры»
Повтор пароля	совпадает с паролем	«Пароли не совпадают»
Согласие	флажок установлен	«Нужно согласие, чтобы создать аккаунт»
Почта (повторно)	не зарегистрирована ранее	«Пользователь с почтой … уже зарегистрирован»

Проверка реализована стандартными средствами Flutter [18]. Поля размещены внутри виджета Form с ключом GlobalKey<FormState>; у каждого поля TextFormField задан параметр validator — метод класса Validators, который возвращает текст ошибки или null, если значение верное. При нажатии «Создать аккаунт» вызывается _formKey.currentState!.validate(): Flutter проверяет все поля сразу и выводит сообщения под неверными (рисунок 3, б). После первой попытки включается режим AutovalidateMode.onUserInteraction, и сообщения исчезают, как только пользователь исправит ввод. Флажок согласия проверяется отдельно, так как не является полем формы.

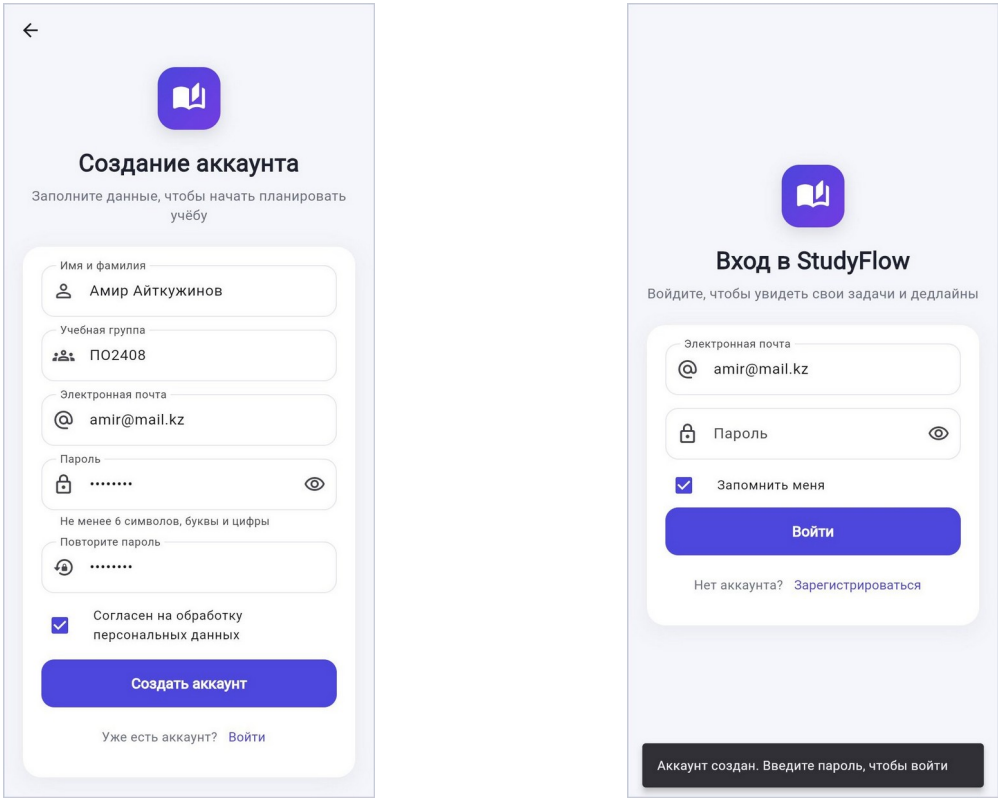


а) б)

Рисунок 3 – Экран регистрации: а – пустая форма; б – сообщения о неверно введённых данных

Если все данные верны, метод store.register проверяет, не занята ли почта, создаёт пользователя, добавляет ему четыре примера задач (чтобы главная страница не была пустой при первом входе) и сохраняет данные. Экран регистрации закрывается вызовом Navigator.pop(context, email) и возвращает почту нового пользователя на экран входа (рисунок 4).

В регистрацию добавлено собственное поле «Учебная группа». Приложение предназначено для студентов, и группа — их основной учебный признак: она отображается в профиле и позволяет в дальнейшем реализовать общий список заданий для всей группы. Регулярное выражение принимает латиницу, кириллицу, включая буквы казахского алфавита, и сохраняет группу в верхнем регистре, поэтому ввод «по2408» превращается в «ПО2408».



а) б)

Рисунок 4 – Завершение регистрации: а – заполненная форма; б – экран входа с подставленной почтой

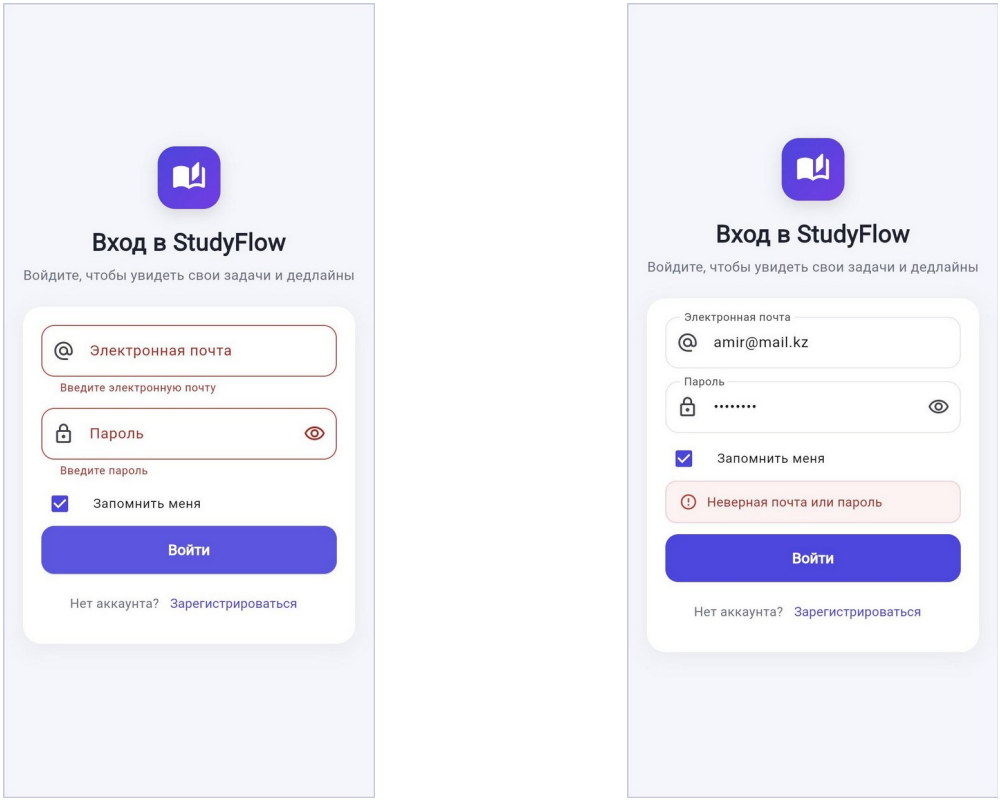
2.6 Экран авторизации и связь с регистрацией

Экран авторизации (рисунок 5) содержит поля почты и пароля, флажок «Запомнить меня», кнопку «Войти» и ссылку «Зарегистрироваться» для перехода к регистрации. Вход выполняется в три шага:

1) проверка формы — пустые поля и формат почты (рисунок 5, а);

2) проверка учётных данных методом store.login: пользователь ищется по почте без учёта регистра и пробелов, затем сравнивается пароль; при ошибке над кнопкой выводится сообщение «Неверная почта или пароль» (рисунок 5, б);

3) при успешном входе вызывается pushNamedAndRemoveUntil, открывается главная страница, а экраны входа удаляются из истории.



а) б)

Рисунок 5 – Экран авторизации: а – проверка пустых полей; б – неверная почта или пароль

Связь регистрации и авторизации обеспечивает общее хранилище AppStore: зарегистрированный пользователь сразу попадает в словарь пользователей и сохраняется в файл, поэтому войти можно сразу после регистрации, после выхода из аккаунта и после перезапуска приложения. Почта, которую вернул экран регистрации, подставляется в поле входа, а старые сообщения об ошибках сбрасываются методом reset() формы.

Состояние входа обрабатывается так: почта вошедшего пользователя хранится в поле _currentEmail. Если включён флажок «Запомнить меня», она записывается в файл как сохранённая сессия, и при следующем запуске экран заставки сразу открывает главную страницу. Если флажок снят, сессия в файл не попадает и после перезапуска потребуется войти заново.

2.7 Главная страница и навигация

Главная страница (рисунок 6) содержит:

– текущую дату и приветствие, зависящее от времени суток («Доброе утро», «Добрый день», «Добрый вечер»), с именем пользователя;

– аватар с инициалами, нажатие на который открывает профиль;

– карточку прогресса: «2 из 4 выполнено», процент выполнения, полосу прогресса и счётчики активных, просроченных и выполненных задач;

– кнопки быстрого доступа «Новая задача», «Все задачи» и «Мой профиль»;

– блок «Ближайшие дедлайны» с тремя ближайшими невыполненными задачами и ссылкой «Все»;

– «Совет дня» по организации учёбы, который меняется в зависимости от дня недели.

Все показатели вычисляются из списка задач пользователя при каждом построении экрана. Оболочка HomeShell обёрнута в ListenableBuilder, который подписан на хранилище, поэтому после добавления или отметки задачи главная страница обновляется автоматически.

В нижней части экрана расположена панель навигации с тремя разделами: «Главная», «Задачи», «Профиль». Текущий раздел выделен цветом, кнопка добавления задачи показывается только в разделе «Задачи». Каждая кнопка на главной странице открывает соответствующий раздел или экран.

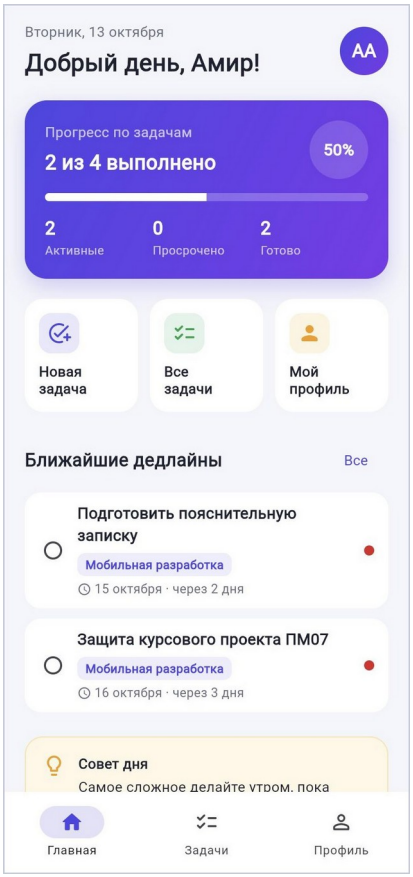


Рисунок 6 – Главная страница

2.8 Основной раздел «Мои задачи»

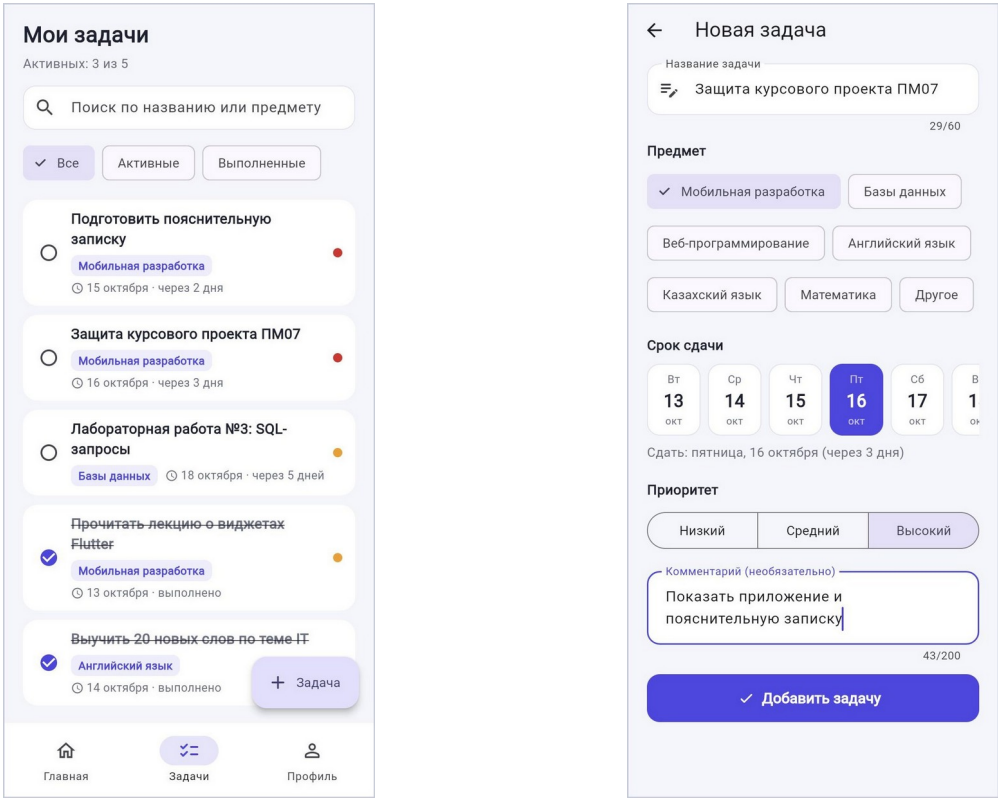
Основное действие пользователя — добавить учебную задачу и отслеживать её выполнение. Раздел «Мои задачи» (рисунок 7, а) показывает список задач, отсортированный так: сначала невыполненные по возрастанию срока, при одинаковом сроке — более важные выше, выполненные — в конце списка.

Карточка задачи содержит круглый флажок выполнения, название (у выполненной задачи оно зачёркнуто), метку предмета, дату и относительный срок («завтра», «через 3 дня», «просрочено на 2 дня»), а также цветную точку приоритета. Срок выделяется цветом: красным — если он прошёл, оранжевым — если наступает сегодня или завтра.

Над списком расположены строка поиска по названию и предмету и фильтры «Все», «Активные», «Выполненные». Удалить задачу можно смахиванием влево (виджет Dismissible) с возможностью отменить удаление во всплывающем сообщении или кнопкой на экране редактирования с

подтверждением. Если подходящих задач нет, выводится пояснение, что делать дальше.

Форма задачи (рисунок 7, б) открывается кнопкой «Задача» или нажатием на карточку. Она содержит название (обязательно, от 3 до 60 символов), выбор предмета из семи вариантов, ленту дат на 30 дней вперёд с русскими названиями дней недели и месяцев, переключатель приоритета и необязательный комментарий. Лента дат сделана вместо стандартного календаря, потому что без подключения пакета локализации он отображается на английском языке. В режиме редактирования добавляются переключатель «Задача выполнена» и кнопка удаления.



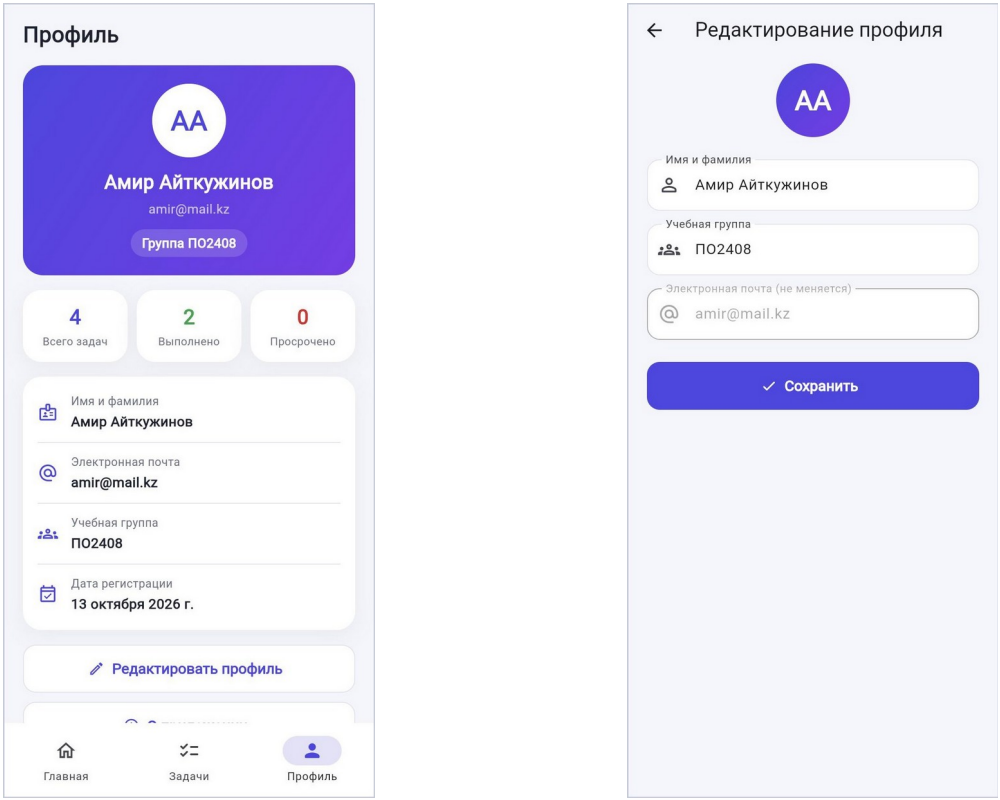
а) б)

Рисунок 7 – Основной раздел: а – список задач; б – добавление задачи

2.9 Профиль и выход из аккаунта

Раздел профиля (рисунок 8, а) показывает аватар с инициалами, имя, почту и учебную группу, статистику («Всего задач», «Выполнено», «Просрочено»), карточку с данными пользователя и датой регистрации, кнопки «Редактировать профиль», «О приложении» и «Выйти из аккаунта».

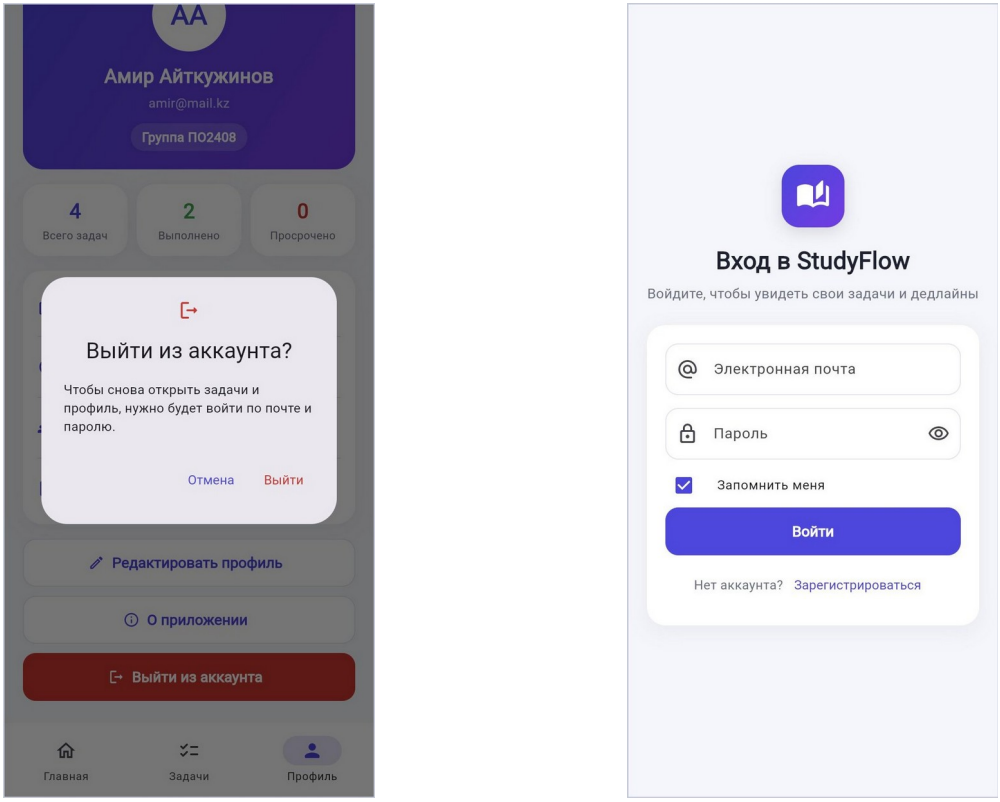
Кнопка «Редактировать профиль» открывает отдельный экран (рисунок 8, б), где можно изменить имя и группу; при этом используются те же правила проверки, что и при регистрации. Почту изменить нельзя, так как она служит логином. Кнопка «Сохранить» обновляет данные и возвращает на профиль с сообщением «Профиль обновлён», стрелка «Назад» возвращает без сохранения.



а) б)

Рисунок 8 – Профиль: а – данные пользователя; б – редактирование профиля

При нажатии «Выйти из аккаунта» появляется окно подтверждения (рисунок 9, а). После подтверждения метод store.logout удаляет почту вошедшего пользователя и сохранённую сессию, затем вызывается pushNamedAndRemoveUntil(AppRoutes.login, (route) => false) — открывается экран авторизации, а вся история экранов очищается (рисунок 9, б). Вернуться в закрытые разделы кнопкой «Назад» невозможно, а маршрут «/home» без вошедшего пользователя также ведёт на экран авторизации.



а) б)

Рисунок 9 – Выход из аккаунта: а – подтверждение выхода; б – экран авторизации после выхода

2.10 Тестирование приложения

Тестирование проводилось тремя способами:

– статический анализ командой flutter analyze — замечаний нет;

– автоматические модульные тесты (flutter test) для хранилища и правил проверки: регистрация, повторная почта, неверный пароль, сохранение и восстановление сессии, склонение слов — все тесты пройдены;

– ручное тестирование полного сценария работы в веб-сборке с размером экрана смартфона (результаты приведены в таблице 8).

Таблица 8 – Результаты ручного тестирования

№	Действие	Ожидаемый результат	Итог
1	Запуск приложения	заставка, затем экран входа	пройден
2	«Войти» с пустыми полями	сообщения под полями	пройден
3	Регистрация с неверными группой, почтой, паролем	сообщения под каждым полем	пройден
4	Регистрация без согласия	подсказка о согласии	пройден
5	Регистрация с занятой почтой	сообщение «уже зарегистрирован»	пройден
6	Верная регистрация	возврат на вход, почта подставлена	пройден
7	Вход с неверным паролем	«Неверная почта или пароль»	пройден
8	Вход с верными данными	открывается главная	пройден
9	Добавление задачи	задача в списке, счётчики обновлены	пройден
10	Отметка выполнения	задача зачёркнута, прогресс вырос	пройден
11	Поиск и фильтр «Активные»	показаны только подходящие задачи	пройден
12	Удаление задачи	подтверждение, задача удалена	пройден
13	Редактирование профиля	новые данные в профиле	пройден
14	Выход из аккаунта	экран входа, возврата в профиль нет	пройден
15	Перезапуск с «Запомнить меня»	сразу открывается главная	пройден

В ходе тестирования были найдены и исправлены ошибки:

– после добавления задачи список в разделе «Мои задачи» не обновлялся, так как вкладка была создана как константный виджет и не перестраивалась; исправлено подпиской вкладки на хранилище через ListenableBuilder;

– длинные сообщения об ошибках обрезались многоточием; исправлено параметром errorMaxLines: 2;

– после регистрации на экране входа оставались старые сообщения об ошибках; исправлено сбросом формы методом reset();

– при редактировании задачи выбранная дата могла оказаться за пределами видимой части ленты дат; исправлено начальной прокруткой ленты к выбранному дню.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработано мобильное приложение StudyFlow — планировщик учебных задач для студентов колледжа. Все поставленные задачи выполнены.

Проанализирована предметная область: определены характеристики учебной задачи и проблемы, с которыми сталкивается студент при планировании. Рассмотрены особенности мобильных приложений и принципы их разработки, проведено сравнение с приложениями Google Tasks, Microsoft To Do, Todoist и TickTick, обоснован выбор фреймворка Flutter и языка Dart.

Спроектированы структура приложения, модель данных и схема навигации. Реализованы экран запуска с анимацией, регистрация с проверкой всех полей и добавленным полем «Учебная группа», авторизация с обработкой неверного входа, сохранение пользователей и сессии, главная страница с прогрессом и ближайшими сроками, нижняя навигация, основной раздел «Мои задачи» с поиском, фильтрами, добавлением, редактированием и удалением задач, профиль с редактированием данных и выход из аккаунта с очисткой истории экранов.

Приложение протестировано статическим анализатором, модульными тестами и вручную по полному сценарию — от запуска до выхода; найденные ошибки исправлены. Таким образом, цель проекта достигнута.

Возможные направления развития приложения:

– уведомления о приближающихся сроках сдачи;

– облачная синхронизация и общий список заданий для учебной группы;

– хранение данных в базе SQLite и хранение паролей в виде хеша;

– тёмная тема оформления и казахская локализация интерфейса;

– виджет для рабочего стола смартфона со списком ближайших сроков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. О персональных данных и их защите : Закон Республики Казахстан от 21 мая 2013 года № 94-V [Электронный ресурс]. – URL: https://adilet.zan.kz/rus/docs/Z1300000094 (дата обращения: 05.10.2026).

2. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – М. : Стандартинформ, 2017.

3. Купер, А. Алан Купер об интерфейсе. Основы проектирования взаимодействия / А. Купер, Р. Рейман, Д. Кронин ; пер. с англ. – СПб. : Символ-Плюс, 2009. – 688 с.

4. Круг, С. Не заставляйте меня думать. Веб-юзабилити и здравый смысл / С. Круг. – 3-е изд. – М. : Эксмо, 2017. – 256 с.

5. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – СПб. : Питер, 2019. – 464 с.

6. Соколова, В. В. Разработка мобильных приложений : учебное пособие для вузов / В. В. Соколова. – М. : Юрайт, 2022. – 175 с.

7. Bailey, T. Flutter for Beginners / T. Bailey, A. Biessek. – 2nd ed. – Birmingham : Packt Publishing, 2021. – 370 p.

8. Napoli, M. L. Beginning Flutter: A Hands On Guide to App Development / M. L. Napoli. – Indianapolis : John Wiley & Sons, 2019. – 528 p.

9. Windmill, E. Flutter in Action / E. Windmill. – Shelter Island : Manning Publications, 2020. – 368 p.

10. Core app quality guidelines [Электронный ресурс] // Android Developers. – URL: https://developer.android.com/docs/quality-guidelines/core-app-quality (дата обращения: 06.10.2026).

11. Human Interface Guidelines [Электронный ресурс] // Apple Developer. – URL: https://developer.apple.com/design/human-interface-guidelines (дата обращения: 06.10.2026).

12. Dart documentation [Электронный ресурс]. – URL: https://dart.dev/language (дата обращения: 07.10.2026).

13. Digital 2026: Kazakhstan [Электронный ресурс] // DataReportal. – URL: https://datareportal.com/reports/digital-2026-kazakhstan (дата обращения: 10.10.2026).

14. Flutter documentation [Электронный ресурс]. – URL: https://docs.flutter.dev (дата обращения: 07.10.2026).

15. Simple app state management [Электронный ресурс] // Flutter documentation. – URL: https://docs.flutter.dev/data-and-backend/state-mgmt/simple (дата обращения: 07.10.2026).

16. Material Design 3 [Электронный ресурс]. – URL: https://m3.material.io (дата обращения: 06.10.2026).

17. Nielsen, J. 10 Usability Heuristics for User Interface Design [Электронный ресурс] / J. Nielsen // Nielsen Norman Group. – URL: https://www.nngroup.com/articles/ten-usability-heuristics/ (дата обращения: 06.10.2026).

18. Build a form with validation [Электронный ресурс] // Flutter documentation. – URL: https://docs.flutter.dev/cookbook/forms/validation (дата обращения: 08.10.2026).

ПРИЛОЖЕНИЕ А. Фрагменты исходного кода

Ниже приведены ключевые фрагменты файла lib/main.dart: проверка вводимых данных, регистрация, авторизация и выход из аккаунта. Полный исходный код представлен в файле проекта.

Листинг А.1 – Класс Validators: проверка вводимых данных

```
class Validators {
  static final RegExp _email = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)*\.[A-Za-z]{2,}$');
  static final RegExp _group =
      RegExp(r'^[A-Za-zA-Яа-яЁёӘәҒғҚқҢңӨөҰұҮүҺһIi]{1,5}-?\d{3,4}$');
  static final RegExp _letters = RegExp(r'[A-Za-zA-Яа-яЁёӘәҒғҚқҢңӨөҰұҮүҺһIi]');
  static final RegExp _digits = RegExp(r'\d');


  static String? name(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Введите имя и фамилию';
    if (text.length < 2) return 'Слишком короткое имя';
    if (_digits.hasMatch(text)) return 'Имя не должно содержать цифры';
    return null;
  }


  static String? group(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Введите учебную группу';
    if (!_group.hasMatch(text)) return 'Формат: буквы и цифры, например ПО2408';
    return null;
  }


  static String? email(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Введите электронную почту';
    if (!_email.hasMatch(text)) return 'Неверный формат почты, пример: name@mail.kz';
    return null;
  }


  /// Пароль при регистрации: не короче 6 символов, есть буквы и цифры.
  static String? newPassword(String? value) {
    final text = value ?? '';
    if (text.isEmpty) return 'Введите пароль';
    if (text.length < 6) return 'Пароль должен быть не короче 6 символов';
    if (!_letters.hasMatch(text) || !_digits.hasMatch(text)) {
      return 'Пароль должен содержать буквы и цифры';
    }
    return null;
  }


  /// Пароль при входе: достаточно, чтобы поле не было пустым.
  static String? password(String? value) {
    if (value == null || value.isEmpty) return 'Введите пароль';
    return null;
  }


  static String? taskTitle(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Введите название задачи';
    if (text.length < 3) return 'Название должно быть не короче 3 символов';
    return null;
  }
}
```

Листинг А.2 – Регистрация, авторизация и выход в классе AppStore

```
/// Регистрация. Возвращает текст ошибки или null, если всё прошло успешно.
Future<String?> register({
  required String name,
  required String group,
  required String email,
  required String password,
}) async {
  final key = email.trim().toLowerCase();
  if (_users.containsKey(key)) {
    return 'Пользователь с почтой $key уже зарегистрирован';
  }
  _users[key] = AppUser(
    name: name.trim(),
    email: key,
    password: password,
    group: group.trim().toUpperCase(),
    registeredAt: DateTime.now(),
  );
  _tasks.addAll(_starterTasks(key));
  await _commit();
  return null;
}

/// Авторизация. Возвращает текст ошибки или null при успешном входе.
Future<String?> login({
  required String email,
  required String password,
  required bool remember,
}) async {
  final user = _users[email.trim().toLowerCase()];
  // Не уточняем, что именно неверно (почта или пароль), — так безопаснее.
  if (user == null || user.password != password) {
    return 'Неверная почта или пароль';
  }
  _currentEmail = user.email;
  _rememberMe = remember;
  await _commit();
  return null;
}

/// Выход: сессия удаляется, для закрытых разделов нужен повторный вход.
Future<void> logout() async {
  _currentEmail = null;
  _rememberMe = false;
  await _commit();
}
```

Листинг А.3 – Обработка входа на экране авторизации

```
  Future<void> _submit() async {
    FocusScope.of(context).unfocus();
    setState(() {
      _submitted = true;
      _error = null;
    });
    // 1) Проверка формы: пустые поля и неверный формат почты.
    if (!_formKey.currentState!.validate()) return;


    // 2) Проверка почты и пароля по сохранённым пользователям.
    setState(() => _loading = true);
    final error = await store.login(
      email: _emailController.text,
      password: _passwordController.text,
      remember: _remember,
    );
    if (!mounted) return;
    setState(() => _loading = false);
    if (error != null) {
      setState(() => _error = error);
      return;
    }
    // 3) Успешный вход: открываем главную и убираем экраны входа из истории,
    //    чтобы кнопка «Назад» не возвращала на авторизацию.
    Navigator.pushNamedAndRemoveUntil(context, AppRoutes.home, (route) => false);
  }
```

Листинг А.4 – Выход из аккаунта с подтверждением

```
/// Выход из аккаунта с подтверждением.
Future<void> confirmLogout(BuildContext context) async {
  final confirmed = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      icon: const Icon(Icons.logout_rounded, color: kDanger),
      title: const Text('Выйти из аккаунта?'),
      content: const Text(
        'Чтобы снова открыть задачи и профиль, нужно будет войти по почте и паролю.',
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Отмена'),
        ),
        TextButton(
          style: TextButton.styleFrom(foregroundColor: kDanger),
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Выйти'),
        ),
      ],
    ),
  );
  if (confirmed != true || !context.mounted) return;
  await store.logout();
  if (!context.mounted) return;
  // Удаляем всю историю экранов: «Назад» не вернёт в закрытые разделы.
  Navigator.pushNamedAndRemoveUntil(context, AppRoutes.login, (route) => false);
}
```